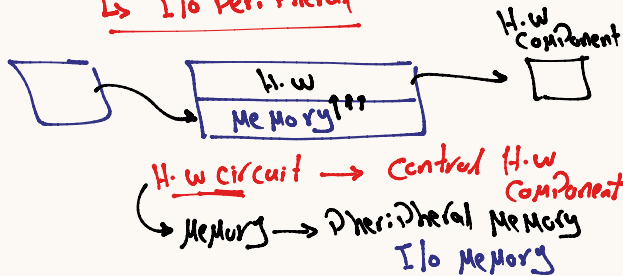


* Revision

↳ I/O Peripheral



* Input/Output

↳ output → Mc will be provide Power

Vcc High
* led * buzzer, lcd, motor,
Gnd. Low

↳ Input → Mc Read the H.W. Component state

→ Button, Keypad, Disital sensor

Digital Input out Put Peripheral (Dio)

↳ to control on digital H.W. Component

↳ (0,1)

↳ digital H.W. Component:

↳ led ↳ 7-segment ↳ Keypad
↳ Button ↳ lcd ↳ Buzzer
↳ Motor ↳ digital sensor (IR)

* to understand Dio must be understand

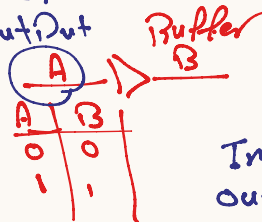
① Dio H.W. circuit (AVR)

② Dio Memory

① Dio H.W. Circuit

* ① Buffer

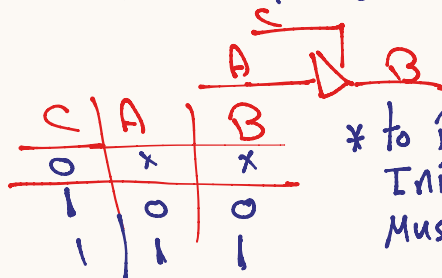
↳ electronics Component take Input value provide output



* Isolate between Input circuit output circuit

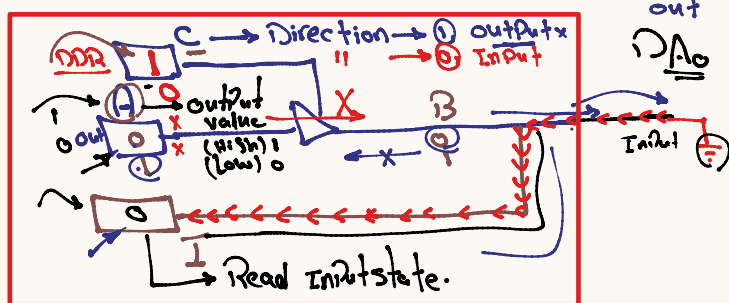
* ② tri: state Buffer

↳ it's Buffer but with control.



* to pass value from Input to output must be C be 1

Dio Circuit over view



* Dio

↳ Direction → output / Input

↳ if Direction as output

↳ High / Low

↳ if Direction as Input

↳ Read Input value

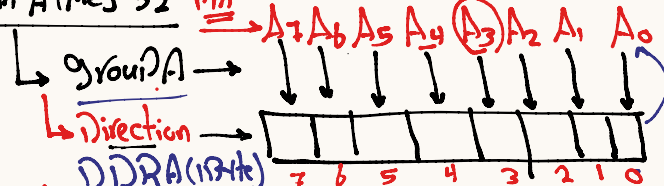
Notes:

* in Atmega32 MCU → No of Pin will be work as Dio → 32 Pin

* For each pin we need 3 bit from memory to control.

* total we need 96 bit to control on 32 pin

* in Atmeg 32 Pin



↳ Direction → DDRA (13 bit)

↳ Output value → PORTA (13 bit)

↳ Input → PINA

Group A

↳ DDRA → to select direction (Input/output)

↳ PORTA → to select output value (High/Low)

↳ PINA → to get the Input value

Group B

DDRB, PORTB, PINB

Group C

DDRC, PORTC, PINC

Group D

DDRD, PORTD, PIND

* Dio Memory

→ 12 Register → I/O Register

Direction

DDRA
DDRB
DDRC
DDRD

Output value

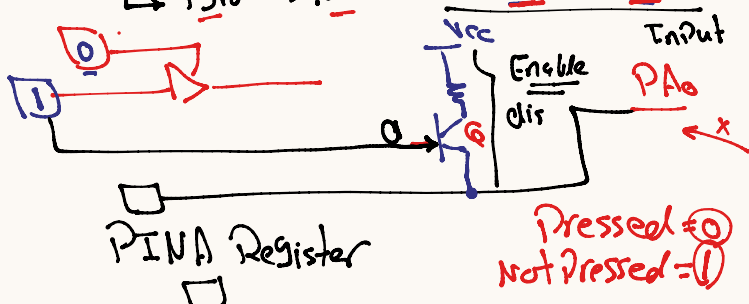
PORTA
PORTB
PORTC
PORTD

Input

PINA
PINB
PINC
PIND

* Input circuit

→ Dio → AVR → Internal Pull up?



Internal Pullup

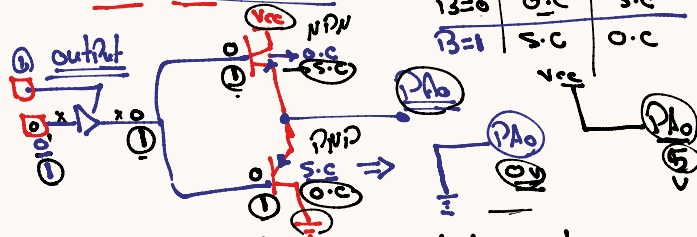
Enable for Internal Pullup

→ ① Direction → Input
(DDR) → 0
→ ② write → High
(PORT) → 1

* Advance topic

→ circuit → output circuit

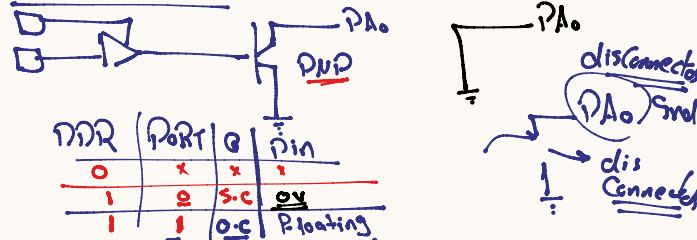
① Push Pull circuit



* → Push Pull

DDR	PORT	Q1	Q2	Pin
0	x	x	x	x
1	0	o.c	s.c	0V
1	1	s.c	o.c	Vcc

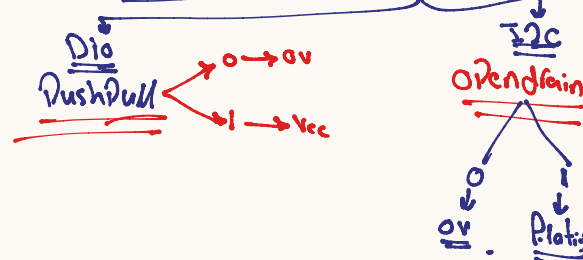
② Open drain



in Atmega32

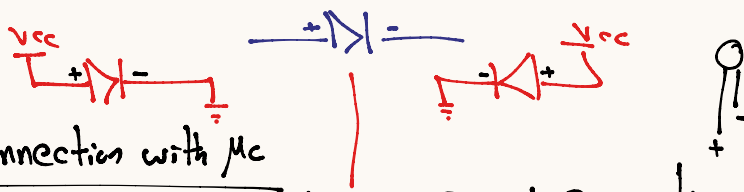
→ all Pin → output circuit Push Pull
→ 32

→ in I2C Pin → have 2 Pin

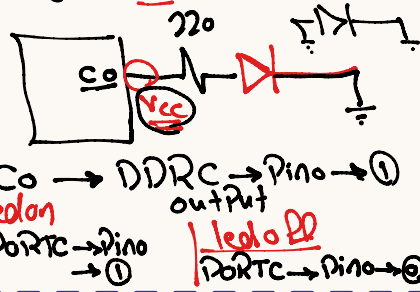


* H.w Component

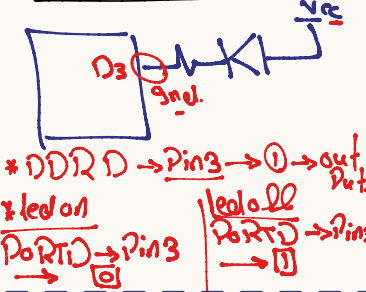
→ led → lighting emitting Diode



① Source Connection



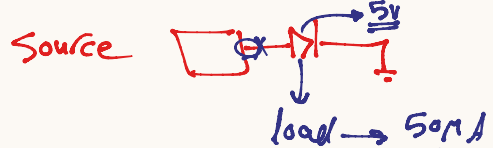
② Sink Connection



* who is Better ?

* Pin of Atmega32 will provide
Max Power → 5V / 40mA

Recommend 20: 25mA

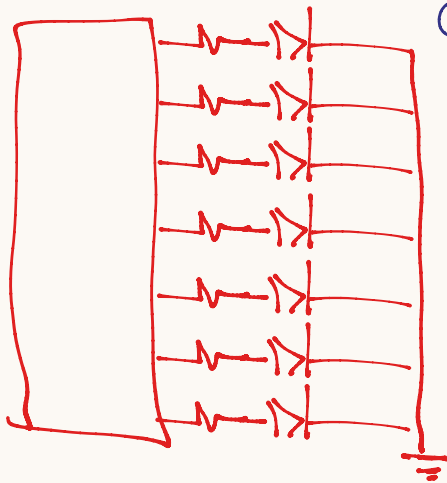


in I/O kit

→ Source Connection



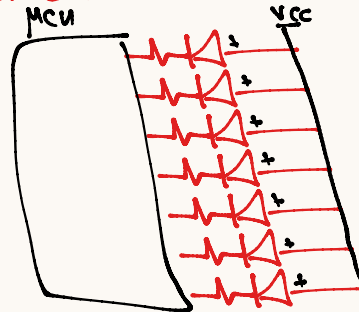
Connect for 7 led → Source Connection



Common: Ground
Common Cathode

led on → PORT → ①
led off → PORT → ②

Connection for 7-lead with Sink Connection



* Common VCC
* Common Anode
led on → PORT → ①
led off → PORT → ②

* 7-Segment

↳ Display Component

↳ led → Notification
↳ segment → Number

* Segment

↳ 7-Segment

7-lead

Common Cathode
Common Anode

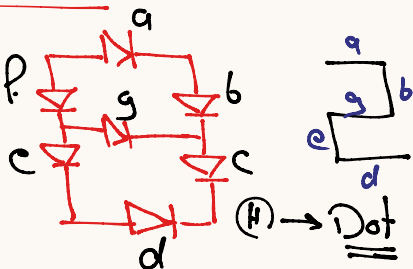
Display No
from 0 → 9

↳ Draw No
Shape

→ Block Diagram

MC → a, b, c, d
e, f, g, h

COM → VCC (Anode)
Ground (Cathode)



(H) → Dot

Common Cathode

led on → 1
led off → 0

H g f e d c b a
7 6 5 4 3 2 1 0

0 1 0 1 1 0 1 1

PORT → 0b01011011

0x5B

Common Anode

led on = 0
led off = 1

H g f e d c b a
7 6 5 4 3 2 1 0
1 0 1 0 0 1 0 0

PORT → 0b10100100

0x:A4

* Notes

↳ 7-segment type

↳ Common Cathode
↳ Anode

↳ Connection

H g f e d c b a
7 6 5 4 3 2 1 0
a b c d e f g h
7 6 5 4 3 2 1 0

* Switch

↳ it's component connect between 2
Point
↳ have two state
↳ Connect
↳ dis Connect

* switch type

↳ Mechanical switch

↳ need human power
to convert from state to
another

↳ Electrical switch

↳ convert between state
by electricity

* Mechanical Switch

↳ Normal Switch :: After human Power Save state.

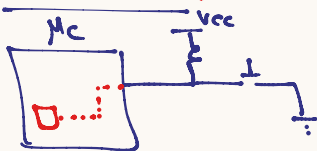
↳ Push Button
↳ Save state when you Provide Power / Power loss Return to default state. → Spring



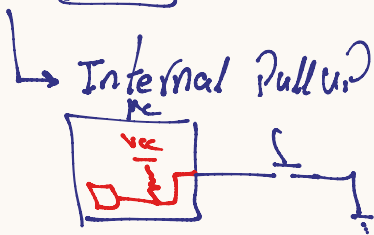
Switch Connection

① Pull up Resistor

↳ External Pull up



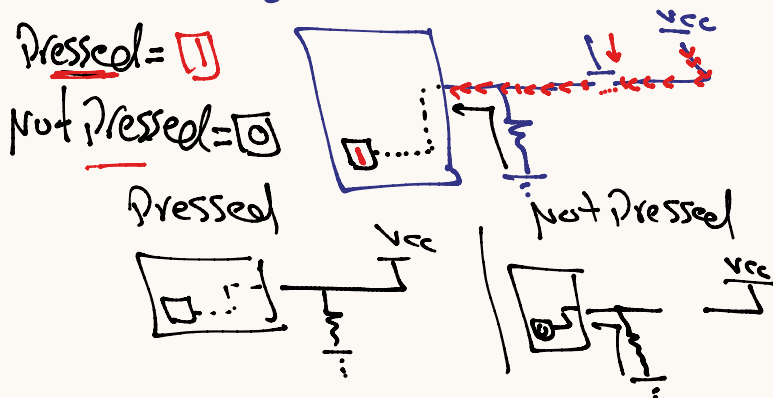
Pressed ⇒ 0
Not Pressed ⇒ 1



Must Enable Internal Pull up
Pressed ⇒ 0
Not Pressed ⇒ Vcc

② Pull down Resistor

↳ External only (AVR)



* In Embedded System

① Code type

↳ ① Superloop system

* Initialization inside before loop

* Application inside main in loop

```
void main()
{
  // Direction
  while(1) → Superloop
  {
    // Application
  }
}
```

* Waiting type

↳ ① Blocking wait

↳ For loop empty → delay_ms (1000 to 2000) ms
↳ Blocked

↳ ② timer wait

↳ ③ OS wait

